Mini Red-Team Generator (Qwen3-0.6B + LoRA) Writeup

I intentionally do not ship raw generated examples in the submission artifacts. The evaluation artifacts are redacted by default to keep the repo safe to share. Reviewers can reproduce locally and inspect generated samples in a controlled environment.

1 How I approached it with a small dataset (and why)

1.1

Given the one-day constraint, I optimized for:

- A working end-to-end loop quickly (data -> train -> generate -> score -> evaluate).
- Control knobs that make it obvious whether the model is following intent.
- A baseline comparison that answers "did fine-tuning actually do anything?".

1.2 Model and training choice

- Base model: `Qwen/Qwen3-0.6B`
- Adaptation: LoRA SFT (PEFT)

Why I picked this:

- 0.6B is big enough to produce coherent completions, but still practical to fine-tune on a single GPU.
- LoRA is the fastest way to produce a reproducible behavior delta without pulling in preference optimization and reward modeling complexity.

1.3 Data strategy: small, diverse, and contract-driven

The most common failure mode in red-team generators is narrow coverage (everything looks like one type of insult).

So I use multiple public datasets and normalize them into a single contract:

- L1: `safe` / `abusive`
- L2: 8 risk categories: `insult`, `threat`, `identity_attack`, `obscene_profanity`, `sexual_explicit`, `harassment`, `hate_speech`, `other_abuse`

In the current run, `prepare_data.py` builds ~28k normalized rows across sources, then training uses only L1=`abusive`. With the current mapping and sampling, that yields ~10k abusive training rows (config-capped by `max_train_samples`).

Why this works in a "small dataset" setting:

- you get coverage across different abuse types without needing to scale to millions of rows,
- it forces you to deal with label mismatch early, and
- it lets you keep the training stable and fast enough to iterate.

1.4 Conditioning: make generation controllable, not random

I treat generation as "conditional synthesis" rather than open-ended creative writing.

Every request includes explicit control tokens:
`[RISK=<L2>] [SEV=<low|mid|high>] [STYLE=<direct|sarcastic|taunt>]`

That gives you:

- a concrete target for evaluation ("did it match the risk I asked for?"),
- a way to probe edge cases by sweeping conditions, and
- a practical interface you could hand to a trust & safety analyst (or a test harness).

1.5 Practical guardrails (chain-of-thought artifacts)

One common nuisance is models emitting reasoning tags instead of the requested output. I prevent that in three places:

- prompt phrasing asks for direct output,
- decoding blocks those tokens via `bad_words_ids`,
- post-processing + rejection sampling hard-reject anything that still contains those tags.

2 How I evaluated it (and whether it is better than a generic generator)

I used a deliberately simple evaluation setup that you can run locally and reason about:

- Two open-source scorers (different training data / inductive biases).
- A single aggregated score used for rejection sampling.
- A baseline model run with identical generation settings.

2.1 Scoring and acceptance

Scorers:

- `unitary/toxic-bert` (multi-label; also used to infer a coarse risk label)
- `cardiffnlp/twitter-roberta-base-offensive`

Aggregate:
`S = 0.6 * toxic_agg + 0.4 * offensive_prob`

Acceptance (rejection sampling) gates on:

- score threshold
- minimum length
- exact de-dup + trigram-Jaccard de-dup

I also add a safety valve: if a strict threshold results in too few accepted samples, the scorer can fall back to a quantile-based threshold. This avoids runs that produce no accepted samples and gives you something to audit, while still keeping the run comparable.

2.2 Baseline comparison

The baseline is the same pipeline on the base model without the LoRA adapter:

- same prompts
- same decoding parameters
- same scoring and acceptance logic

The `evaluate.py` script recomputes acceptance for both main and baseline using the same threshold to keep the comparison fair even if the scorer had to adapt thresholds during scoring.

2.3 Results summary (from current artifacts)

From `outputs/reports/eval_report.json` (LoRA) vs `outputs/reports/baseline_report.json` (baseline):

Metric	LoRA	Baseline
Mean risk score	0.1260	0.0886
Harmful hit rate (at used threshold)	0.1042	0.0292
Accepted category coverage	0.75	0.375

Distinct-2	0.8483	0.6795

How I interpret this:

- The adapted model is consistently shifting the score distribution upward (mean score delta).
- It produces a higher fraction of high-scoring samples at the same threshold (hit rate delta).
- It covers more of the taxonomy in its accepted set, which matters for red-teaming.

Is it "good" in an absolute sense? It is a working PoC, but the absolute hit-rate depends heavily on threshold calibration and scorer bias. That is why I include score quantiles in the JSON reports and keep a human audit loop in the pipeline.

2.4 Manual audit loop (human check)

Automated scorers are noisy and can be overconfident. The repo generates a small manual audit sheet (20 rows) to spot obvious failure modes (prompt echoing, condition mismatch, non-English).

The submission version is redacted by default to avoid shipping disallowed text. For a local-only review, set `manual_audit.redact: false` and re-run evaluation.

3 Trade-offs, and how I would communicate them to a team

If I had to summarize this to a team lead in five minutes:

- "We have a controllable generator with measurable acceptance and a baseline comparison. Next step is calibration: per-risk thresholds, better risk inference, and a review workflow."

Key trade-offs:

1. LoRA SFT instead of DPO/RL
   - Pros: fast, stable, reproducible in one day.
   - Cons: less direct preference control; more reliance on training data quality + rejection sampling.
2. Multiple datasets instead of one dataset
   - Pros: better coverage across abuse types.
   - Cons: label semantics mismatch; you need a mapping layer and you inherit some noise.
3. Dual scorers instead of one scorer
   - Pros: reduces single-model evaluator bias.
   - Cons: more runtime and more threshold calibration work.

4. Redaction by default in artifacts
    - Pros: safe to share and submit.
    - Cons: reviewers cannot see raw examples unless they rerun locally.

4 End-to-end runnable solution

Setup:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -r requirements.txt
```

Run the full pipeline:

```
bash scripts/run_end_to_end.sh full
```

Key outputs:

- `outputs/reports/eval_report.json`
- `outputs/reports/baseline_report.json`
- `outputs/reports/comparison_report.json`
- `outputs/reports/manual_audit_sheet.csv` (redacted by default)